`temperature`), the value is the observation we're interested in (say, `2.5`), and the metadata provides context (e.g., `room_42` for location-related context). The unit of the observation can be encoded in the name or provided as metadata. Last but not least, the metric is timestamped, either explicitly by the producer of the metric or at ingest (server side).

Traditionally, semantics would often be encoded in a hierarchical manner, but with the advent of Kubernetes and Prometheus, the label-based, flat organization scheme became more mainstream. This label-based scheme is more flexible and extensible. The following example demonstrates the difference:

```
room_42.sensor_window.temperature 2.5
```

A metric using a hierarchical scheme

```
temperature[location="room_42", sensor="window"] 2.5
```

The same metric, using a label-based scheme. If one wanted to add another aspect, such as the position, with the label-based scheme, it would be as easy as adding a new label (e.g., position="top"), whereas with an hierarchical scheme, one would need to reconsider the entire name.

We usually differentiate between low-level or systems metrics and application-level or business metrics. The former are resource usage metrics, like CPU utilization or allocated main memory, and the latter include things like requests handled per second or revenue generated per user. In general, low-level metrics can often be automatically gleaned, whereas application-level metrics require a human to assist in definition and exposure.

Another aspect of metrics, again popularized by Prometheus, is the pull-versus-push approach for metrics. The gist is that, in a cloud-native setup where one deals with containers or functions that are short lived, the discovery is better centralized and the central entity collects the metrics (or scrapes, as Prometheus folks would say—technically, it means calling an HTTP API endpoint to retrieve metrics) rather than having the applications push the metrics to a central place.

When determining which metrics are important to consider, the community came up with several approaches and recommendations. The following are most notable:

- *The utilization, saturation, errors (USE) method*—Created by Brendan Gregg (see https://www.brendangregg.com/usemethod.html), this method is great for low-level metrics and often used in the context of performance engineering and root cause analysis.
- *The rate, errors, duration (RED) method*—Created by Tom Wilkie (see http://mng.bz/PzJ9), this method focuses on the number of requests served, the number of failed requests, and how long requests take. This is useful for application-level cases.
- *The four golden signals of monitoring*—Established by Google engineers (see http://mng.bz/JgJ0), the four signals are latency, traffic, errors, and saturation, which is like RED and USE combined.